

Práctica 3:

Análisis de temas en documentos



Aplicaciones Avanzadas de la Inteligencia Artificial

Marzo 2025

Máster en Ingeniería Informática

Pablo Hidalgo Delgado. NIA: 100451225

Sergio Caño Amor. NIA: 100429678

Eduardo Enrique Angulo Luna: NIA: 100545176

ÍNDICE

1. Introducción	2
2. Análisis exploratorio de datos (EDA)	2
3. Preproceso	5
3.1 Aleatorización	5
3.2 Limpieza y normalización de los textos	5
4. Modelo supervisado	5
4.1 Vectorización	5
4.2 Construcción del modelo	6
4.3 Experimentación	6
4.4 Resultados obtenidos	7
4.4 Análisis de resultados	8
5. Modelo no supervisado	9
5.1 Construcción del modelo	9
5.2 Experimentación	9
5.3 Resultados obtenidos	10
5.4 Análisis de resultados	12
6. Conclusiones	13

1. Introducción

El objetivo de esta práctica es analizar temas en documentos de noticias en inglés utilizando dos enfoques distintos: un modelo supervisado de clasificación de temas y un modelo no supervisado de modelado de temas.

Para completar la primera tarea, entrenaremos un clasificador multiclase que será capaz de predecir la categoría a la que pertenece un documento, utilizando datos previamente etiquetados en cinco temas distintos. Para evaluar el rendimiento del modelo, emplearemos validación cruzada y lo evaluaremos con el conjunto de test.

En la segunda tarea, aplicaremos técnicas de clustering, ignorando las etiquetas. El objetivo es identificar los principales temas tratados en los documentos, agrupándolos en clústeres según su contenido. Para ello, construiremos un modelo K-Means utilizando embeddings estáticos, aunque exploraremos otras opciones.

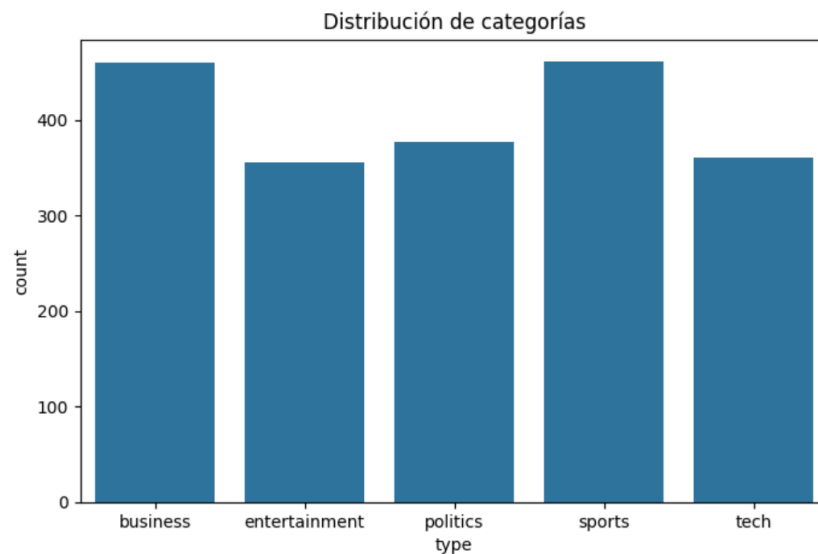
2. Análisis exploratorio de datos (EDA)

Antes de empezar con el preprocesamiento de los datos, realizamos un Análisis Exploratorio de Datos (EDA). De esta manera, investigamos, identificamos y analizamos las características más importantes del conjunto de datos que disponemos. Esto nos sirve para comprender mejor los textos y optimizar la obtención de nuestro modelo.

Por medio de este análisis, observamos que el conjunto se compone de un total de 2015 textos. Además, disponemos de 5 clases diferentes, y cada clase cuenta con un número distinto de textos, concretamente:

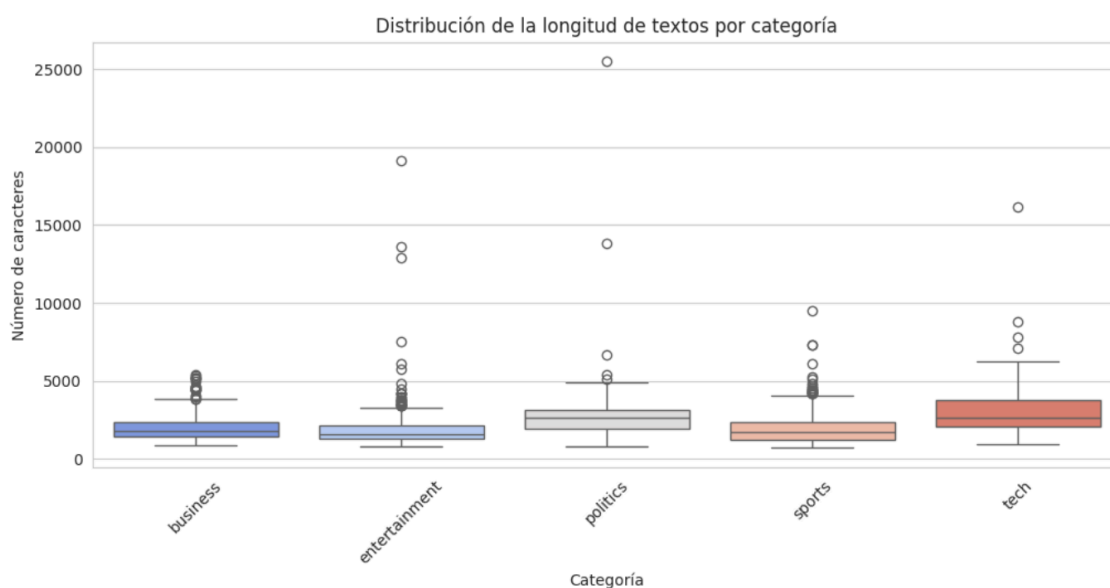
- Categoría *business*: 461 textos
- Categoría *entertainment*: 356 textos
- Categoría *politics*: 377 textos
- Categoría *sports*: 461 textos
- Categoría *tech*: 361 textos

Se puede observar claramente la distribución de clases en la siguiente gráfica:



Analizando la gráfica presentada, consideramos que nuestro conjunto de datos está medianamente balanceado, por lo que no aplicaremos técnicas propias de problemas desbalanceados.

Además, hemos realizado un análisis de la longitud de los textos y el número de palabras por categoría. Observamos que categorías como *tech* y *politics* tienden a tener textos más largos, mientras que *sports* y *entertainment* suelen ser más concisos. Además, notamos una gran variabilidad en la longitud de los textos, con la presencia de valores atípicos (outliers). Esto se refleja en la siguiente gráfica:



3. Preproceso

Previamente a entrenar los modelos, debemos realizar una transformación de los textos para que el modelo construido pueda enfocarse en las características realmente importantes. Las transformaciones de datos que realizamos para ambos modelos son las siguientes:

3.1 Aleatorización

En nuestro caso, es importante aleatorizar los datos ya que los textos se encuentran ordenados por categoría, por lo que el modelo podría captar patrones relacionados con este orden.

3.2 Limpieza y normalización de los textos

1. Conversión a mayúsculas: todos los textos se transforman a minúsculas para evitar que el modelo trate como palabras diferentes aquellas que sólo difieren por su uso de mayúsculas.
2. Eliminación de signos de puntuación: se eliminan los signos de puntuación y caracteres especiales, ya que no aportan significado semántico y pueden agregar ruido al conjunto de datos.
3. Eliminación de números: los números no son relevantes en este caso, por lo que se eliminan para evitar que el modelo los utilice como características.
4. Eliminación de stopwords en inglés: las stopwords son palabras muy comunes (como "the", "is", "and") que aparecen frecuentemente pero no contribuyen al contenido relevante del texto. Eliminarlas nos permite reducir el ruido y centrar el análisis en palabras clave.
5. Lematización: la lematización convierte cada palabra a su forma base o raíz gramatical. Por ejemplo, palabras como running, ran o runs se reducen a run. Esto reduce la variabilidad y mejora la capacidad del modelo para generalizar.

4. Modelo supervisado

En esta sección se explicará la construcción, entrenamiento, experimentación, evaluación y análisis del modelo supervisado de clasificación de textos.

4.1 Vectorización

Después de haber limpiado y normalizado los textos, los convertimos en una representación numérica que pueda ser utilizada por los algoritmos de aprendizaje automático. Para el modelo supervisado, decidimos utilizar el enfoque de Bag of Words (BoW).

El modelo Bag of Words representa cada documento como un vector de frecuencias de palabras, donde cada dimensión del vector corresponde a una palabra en el vocabulario del conjunto de datos, y su valor es la cantidad de veces que esa palabra aparece en el documento. Este enfoque es simple pero efectivo, ya que convierte el texto en una forma estructurada que los modelos pueden procesar.

Para llevar a cabo esta vectorización, utilizamos el CountVectorizer de la librería scikit-learn. Este vectorizador realiza la tokenización implícita, es decir, automáticamente divide el texto en palabras o tokens, y calcula la frecuencia de cada token en el corpus.

4.2 Construcción del modelo

Para abordar el problema de clasificación multiclase, seleccionamos el modelo Naive Bayes Multinomial, ampliamente utilizado en tareas de clasificación de texto. Este modelo es particularmente adecuado para datos representados mediante el enfoque Bag of Words (BoW), ya que maneja eficientemente características discretas, como las frecuencias de palabras en los textos. Su elección se debe a su eficiencia computacional, su capacidad para manejar datos de alta dimensionalidad y su buen rendimiento en clasificación de texto, incluso con conjuntos de datos limitados.

Como hemos explicado en el apartado anterior, el enfoque Bag of Words nos permite transformar los textos en una representación numérica, donde cada documento se convierte en un vector basado en la frecuencia de aparición de palabras en el vocabulario. Este método es simple pero efectivo para modelar texto, aunque tiene la limitación de no capturar el contexto ni la semántica de las palabras.

Para optimizar el rendimiento del modelo, realizamos una búsqueda de hiperparámetros con GridSearchCV, explorando distintos valores del parámetro de suavizado (*alpha*) y de la opción *fit_prior*, aplicando una validación cruzada de 5 folds para mejorar la generalización.

Además de Naive Bayes Multinomial, hemos considerado otras alternativas, como Regresión Logística y SVM (Support Vector Machines). La Regresión Logística, aunque efectiva para clasificación binaria, puede volverse computacionalmente costosa en problemas multiclase con datos de alta dimensionalidad. Por otro lado, SVM ofrece una gran capacidad de generalización, pero su complejidad computacional aumenta significativamente con conjuntos de datos grandes. Dado el equilibrio entre precisión y eficiencia, descartamos estas opciones en favor de Naive Bayes, que mostró un rendimiento sobresaliente con menor costo computacional.

4.3 Experimentación

Como hemos mencionado anteriormente, llevamos a cabo diversos experimentos ajustando los hiperparámetros clave, como *alpha* y *fit_prior*. El parámetro *alpha* controla el nivel de suavizado en el modelo, ayudando a prevenir la asignación de probabilidad cero a características que no aparecen en los datos de entrenamiento. Por otro lado, *fit_prior* define si los priors de las clases deben ser aprendidos a partir de los datos (True) o si se

deben asumir uniformemente distribuidos (False). Entrenamos y evaluamos un total de 10 modelos con diferentes configuraciones de estos hiperparámetros.

Para seleccionar el mejor modelo, utilizamos accuracy como métrica de evaluación, dado que nuestro conjunto de datos presenta un equilibrio adecuado entre clases. Accuracy nos permite medir la proporción de predicciones correctas sobre el total de ejemplos, proporcionando una evaluación clara y directa del desempeño del clasificador.

Una vez identificado el modelo con mejor rendimiento, lo entrenamos nuevamente utilizando todo el conjunto de datos de entrenamiento y validación combinados. Esto permite que el modelo aproveche una mayor diversidad de ejemplos, mejorando su capacidad de generalización y su desempeño en datos no vistos.

4.4 Resultados obtenidos

Los resultados de los distintos modelos construidos son los siguientes:

grid_search_results_with_metrics

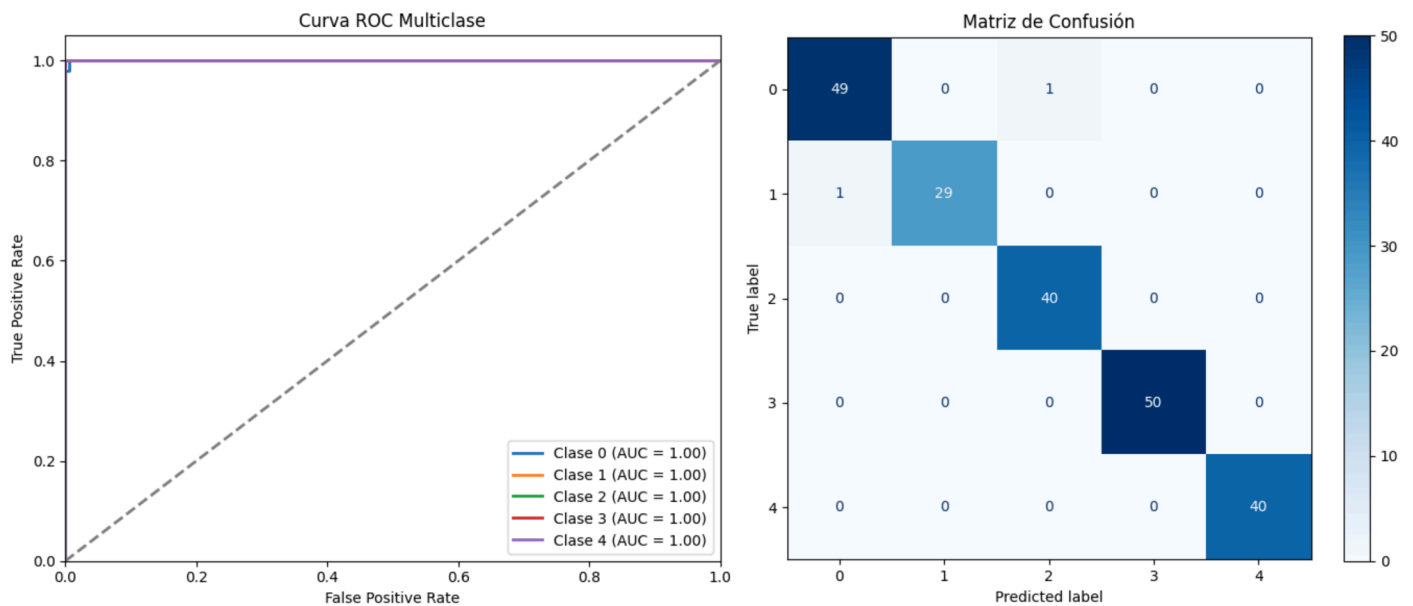
param_alpha	param_fit_prior	mean_val_accuracy	mean_val_precision_macro	mean_val_recall_macro	mean_val_f1_macro	rank_accuracy
0.1	True	0.9737	0.9726	0.97367	0.97285	1
0.1	False	0.9737	0.9726	0.97367	0.97285	1
0.5	True	0.9737	0.97274	0.97359	0.97287	1
0.5	False	0.9737	0.97274	0.97359	0.97287	1
1.0	True	0.97122	0.97041	0.97074	0.97015	5
1.0	False	0.97122	0.97041	0.97074	0.97015	5
5.0	True	0.96328	0.96295	0.96237	0.96152	7
5.0	False	0.96328	0.96295	0.96237	0.96152	7
10.0	False	0.95831	0.95854	0.95683	0.95583	9
10.0	True	0.95732	0.95762	0.95573	0.95479	10

Para seleccionar el modelo óptimo, priorizamos aquel que maximiza el accuracy en el conjunto de validación. Observamos que los cuatro primeros modelos presentan métricas muy similares; sin embargo, identificamos una tendencia en la que el rendimiento tiende a disminuir a medida que el parámetro alpha aumenta. Con base en este análisis, optamos por el modelo con el valor de alpha más bajo.

Así, los hiperparámetros seleccionados para el modelo final son:

- *alpha* = 0.1, *fit_prior* = "True"

Con estos valores, entrenamos el modelo definitivo utilizando el conjunto combinado de entrenamiento y validación. Finalmente, evaluamos su rendimiento en el conjunto de test, obteniendo los siguientes resultados.



model_evaluation_metrics

Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
0.99048	0.99112	0.98933	0.99014

4.4 Análisis de resultados

Tras analizar los experimentos realizados con el conjunto de validación, observamos que, en general, los resultados obtenidos son consistentemente buenos, con unos valores superiores al 95% en todas las métricas obtenidas.

Además, como se ha mencionado antes, identificamos un patrón claro: a medida que el valor de alpha aumenta, el desempeño del modelo tiende a disminuir. Esto indica que un alpha bajo es más adecuado para nuestro problema, ya que permite una mejor adaptación a los datos.

En cuanto al modelo seleccionado (alpha = 0.1, fit_prior = True), los resultados destacan por su alto rendimiento, alcanzando un accuracy del 99%. Al examinar la matriz de confusión, observamos que casi todas las clasificaciones son correctas, con solo dos errores:

- Un texto de la clase business (0) fue incorrectamente clasificado como politics (2).
- Un texto de la clase entertainment (1) fue clasificado erróneamente como business (0).

5. Modelo no supervisado

En esta sección se explicará la construcción, entrenamiento, experimentación, evaluación y análisis del modelo supervisado de clustering de textos.

5.1 Construcción del modelo

Para la construcción del modelo de clustering, decidimos utilizar la representación de los tokens como embeddings estáticos. Exploramos distintos enfoques para realizar esta representación, como GloVe o FastText, pero finalmente elegimos Word2Vec debido a su eficiencia en representar relaciones semánticas de las palabras en un espacio vectorial, y su capacidad para entrenarse rápidamente con grandes volúmenes de datos.

Especificamos que el modelo genere vectores de 100 dimensiones para cada palabra, utilizando una ventana de contexto de 5 palabras y considerando solo las palabras que aparecen al menos 2 veces en el corpus. Word2Vec convierte las palabras en vectores numéricos que capturan sus relaciones semánticas, lo que permite representar las palabras en un espacio vectorial donde palabras similares están cerca unas de otras.

La decisión de usar un embedding estático y no Bag of Words (BoW) como utilizamos en el modelo supervisado anterior, se basó en varias razones. Los embeddings estáticos son capaces de capturar mejor las relaciones semánticas entre palabras, ya que representan cada palabra en un espacio denso y continuo, donde palabras con significados similares están cerca unas de otras. Esto permite que el modelo sea más efectivo al tratar con sinónimos o palabras relacionadas, algo que BoW no puede capturar adecuadamente.

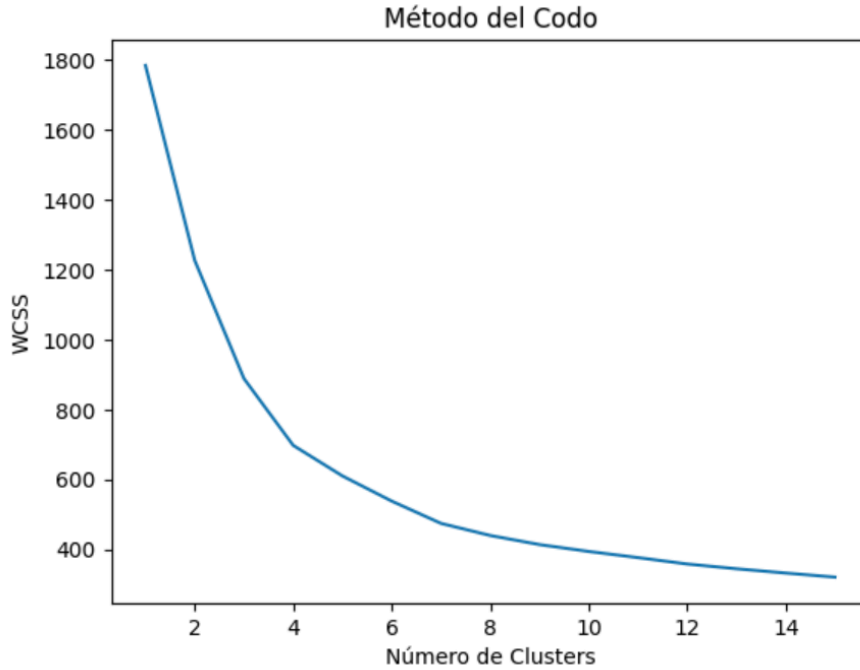
Concretamente, para representar cada documento como un único vector, se promedian los vectores de las palabras de cada texto. Si un documento contenía palabras que no se encontraban en el vocabulario del modelo, asignamos un vector nulo (un vector de ceros). Finalmente, convertimos todos los vectores promediados de los documentos en un array de Numpy, el cual fue utilizado para realizar el clustering posterior.

5.2 Experimentación

En cuanto a los experimentos realizados, aplicamos el “método del codo” para determinar el número óptimo de clusters a utilizar en el algoritmo K-Means.

Para ello, calculamos la inercia (WCSS) para diferentes valores de `n_clusters` (en este caso, de 1 a 15). La inercia mide la suma de las distancias cuadradas dentro de cada cluster, y su valor disminuye a medida que aumentamos el número de clusters.

Posteriormente, obtenemos la gráfica del WCSS en función del número de clusters para observar el punto de codo, que es el lugar donde la reducción de la inercia se vuelve más lenta. Este punto indica el número ideal de clusters para nuestro modelo, ya que agregar más clusters más allá de este punto no mejora significativamente la calidad del clustering.



Como se puede observar, la curva de WCSS presenta una disminución pronunciada hasta aproximadamente $k = 5, 6, 7$ o 8 (donde k es el número de categorías), a partir de donde la reducción se vuelve más gradual. Esto sugiere que la ganancia en cohesión dentro de los clusters es significativa hasta estos valores, pero luego los beneficios de agregar más clusters disminuyen. Dado que no tenemos un punto exacto claramente definido, decidimos construir modelos con estos cuatro valores de k y evaluar métricas como la coherencia y el coeficiente de silueta, lo que nos permitirá seleccionar el número óptimo de clusters basándonos en los resultados obtenidos.

5.3 Resultados obtenidos

Los resultados de los distintos modelos construidos son los siguientes:

metricas_clusters

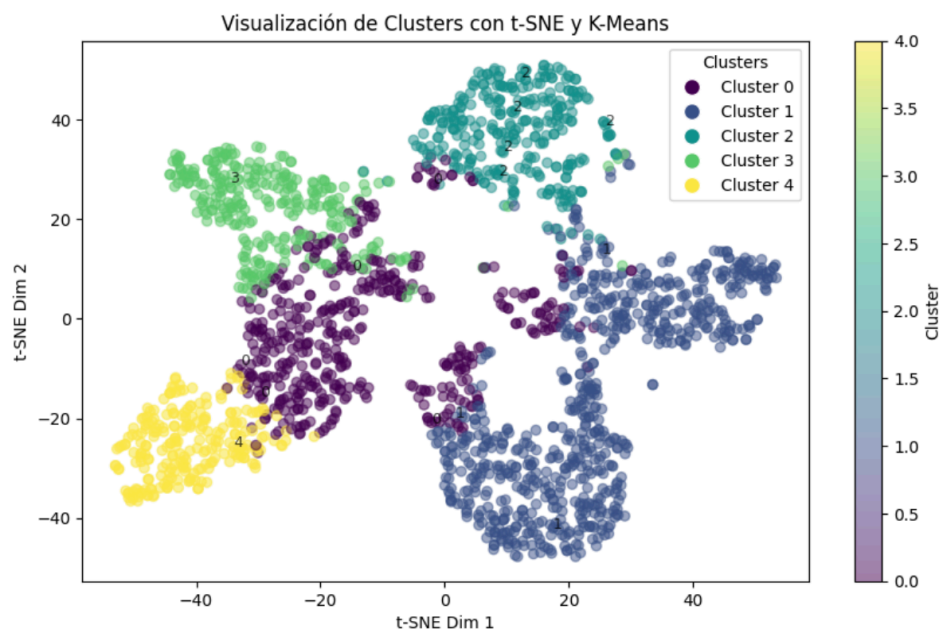
Num_Clusters	Silhouette_Score	Davies_Bouldin_Index	Calinski_Harabasz_Index	Coherence
5	0.29542	1.23654	967.48441	0.29261
6	0.28751	1.24553	930.84418	0.29425
7	0.28681	1.13734	924.49324	0.29128
8	0.26975	1.18138	877.05816	0.29203

En base a los resultados obtenidos, elegimos el modelo con 5 clusters como la mejor opción. Este modelo presenta el mayor coeficiente de silueta (0.2954), lo que indica que los clusters están mejor definidos y separados entre sí en comparación con los demás. Además, su índice de Davies-Bouldin (1.2365) es relativamente bajo, lo que sugiere una menor dispersión dentro de los clusters. También obtiene el mayor índice de Calinski-Harabasz (967.48), lo que indica una buena relación entre la compactación y la separación de los clusters.

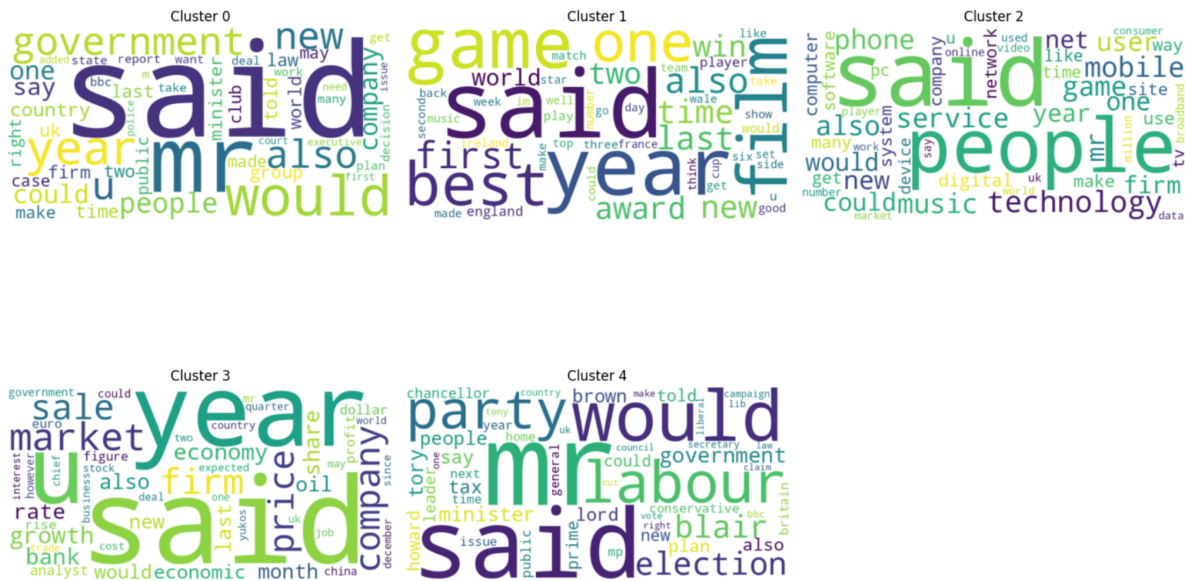
Aunque el modelo con 6 clusters tiene una coherencia ligeramente mayor, su coeficiente de silueta y el índice de Calinski-Harabasz son menores, lo que sugiere que los clusters no están tan bien separados. Por otro lado, los modelos con 7 y 8 clusters muestran una disminución en la coherencia y el coeficiente de silueta, lo que indica que agregar más clusters no mejora significativamente la segmentación y puede introducir una división innecesaria.

Por estas razones, seleccionamos el modelo de 5 clusters como el más adecuado, ya que ofrece una segmentación equilibrada con una mejor separación y cohesión dentro de los grupos.

Para este modelo seleccionado, obtenemos una reducción de dimensionalidad utilizando t-Distributed Stochastic Neighbor Embedding (t-SNE), una técnica de aprendizaje automático que permite proyectar datos de alta dimensión en un espacio bidimensional. Esto nos permite visualizar los clústeres en un gráfico de dispersión, donde cada punto representa un documento y el color indica la asignación de clúster.



También obtenemos la nube de palabras más representativas para cada categoría:



5.4 Análisis de resultados

Analizando la gráfica de dispersión generada, los clústeres están bien definidos, aunque se observa cierta proximidad entre algunos de ellos, lo que sugiere que ciertos textos podrían compartir características similares.

En términos de espaciado y cohesión, se nota que algunos clústeres son más compactos, como el amarillo en la parte inferior izquierda, lo que indica que los textos dentro de este grupo son más homogéneos. Por otro lado, el clúster morado y el azul presentan una mayor dispersión, lo que sugiere una mayor variabilidad interna dentro de estos grupos.

En cuanto a la reducción de dimensionalidad, t-SNE ha agrupado los textos en una estructura con regiones bien separadas, reflejando que K-Means ha capturado patrones diferenciados en los datos originales. No obstante, cabe señalar que t-SNE introduce distorsiones en las distancias, lo que significa que la separación visual observada no siempre representa con exactitud las relaciones en el espacio original de los embeddings.

Por otro lado, observando la nube de palabras más representativas para cada cluster identificado, podemos identificar lo siguiente:

- El Cluster 0 está dominado por términos relacionados con el gobierno y la política, como *government*, *mr*, *year*, *country* y *law*. Esto sugiere que este grupo agrupa noticias sobre temas gubernamentales, decisiones políticas y asuntos legales.
- El Cluster 1 se centra en el ámbito deportivo y de entretenimiento, con palabras clave como *game*, *best*, *win*, *award* y *first*. La presencia de términos asociados a competencias y premios indica que este grupo está conformado por textos sobre eventos deportivos, torneos y reconocimientos dentro del sector del entretenimiento.

- El Cluster 2 se enfoca en tecnología y servicios digitales, con términos como *technology*, *mobile*, *phone*, *service* y *digital*. La fuerte presencia de palabras relacionadas con dispositivos móviles, redes y servicios tecnológicos sugiere que este grupo está compuesto por noticias sobre avances tecnológicos y tendencias en el sector digital.
- El Cluster 3 agrupa textos relacionados con economía y mercados financieros, reflejado en palabras como *market*, *price*, *sale*, *bank* y *economy*. Este conjunto de textos parece abordar fluctuaciones económicas, análisis de mercado y temas financieros de interés general.
- Finalmente, el Cluster 4 muestra un enfoque en política y elecciones, con términos como *labour*, *party*, *election*, *blair* y *government*. La fuerte presencia de nombres y términos asociados a partidos políticos y procesos electorales sugiere que este grupo está compuesto por noticias y artículos sobre política y gobernanza.

En términos de correspondencia con las categorías predefinidas (*business*, *entertainment*, *politics*, *sports* y *tech*), el modelo ha mostrado una segmentación lógica y consistente. Los clusters de business y tech (cluster 2 y cluster 3) presentan una coincidencia muy alta con sus respectivas categorías, reflejando una agrupación precisa. Los clusters de politics están más bien diferenciados entre temas más relacionados con el gobierno y leyes (cluster 0) y otros relacionados con elecciones (cluster 4), lo que indica que el modelo ha capturado matices dentro de esta temática. Sin embargo, se observa cierta superposición entre sports y entertainment (cluster 1), lo que es esperable, dado que los eventos deportivos suelen incluir elementos de entretenimiento.

En general, los resultados sugieren que la clusterización mediante K-Means ha sido efectiva para agrupar textos por su similitud temática, aunque en algunos casos se presentan solapamientos debido a la naturaleza multidimensional de los contenidos.

6. Conclusiones

En resumen, en este trabajo se han desarrollado y evaluado dos modelos de clasificación de textos: uno supervisado y otro no supervisado, con el objetivo de segmentar y agrupar textos en categorías temáticas.

El modelo supervisado, basado en el enfoque de Bag of Words (BoW) y el clasificador Naive Bayes Multinomial, ha demostrado ser altamente eficiente para la tarea de clasificación de textos. Tras realizar una búsqueda de hiperparámetros, se seleccionó el modelo con un valor de $\alpha = 0.1$ y $\text{fit_prior} = \text{True}$, lo que permitió alcanzar un alto rendimiento, con un accuracy del 99%. La matriz de confusión reveló una pequeña cantidad de errores, lo que indica que el modelo es capaz de generalizar bien a datos no vistos.

En cuanto al modelo no supervisado de clustering utilizando K-Means, los resultados mostraron que el número óptimo de clusters era 5 (el mismo que el número de categorías predefinidas), proporcionando la mejor separación y cohesión dentro de los grupos. A través del análisis de la inercia y el coeficiente de silueta, se determinó que este número de clusters era el más adecuado para nuestra tarea. La visualización con t-SNE permitió

observar una buena separación entre los clusters, aunque algunas superposiciones fueron evidentes, lo cual es esperado debido a la naturaleza multidimensional de los textos.

La nube de palabras para cada cluster identificó categorías temáticas consistentes con los dominios de negocio, tecnología, deportes, política y entretenimiento. Sin embargo, se observó cierta superposición entre los clusters de deportes y entretenimiento, lo cual refleja la dificultad inherente en la separación de textos que comparten características comunes, como los eventos deportivos que a menudo se entrelazan con el entretenimiento. Además, el modelo dividió los textos de política en 2 subcategorías.

Finalmente, es importante destacar que, aunque experimentamos con otros modelos durante el desarrollo de este trabajo, decidimos no incluirlos en el informe final, ya que dichos experimentos fueron más orientados a exploraciones internas. Sin embargo, los detalles de estos modelos adicionales se encuentran disponibles en el notebook, en la sección "Modelos Adicionales".